Metodi speciali

Description	Metodi speciali forniscono funzionalità specializzate per le classi

I metodi sono metodi che iniziano e terminano con due trattini bassi (`__`). Questi metodi speciali forniscono funzionalità specializzate per le classi e possono essere sovrascritti per personalizzare il comportamento degli oggetti in diverse situazioni.

`__init__(self, ...)`:

Questo è il costruttore di una classe. Viene chiamato automaticamente quando viene creato un nuovo oggetto della classe. È utilizzato per inizializzare gli attributi dell'oggetto.

```
pythonCopy code
class MyClass:
    def __init__(self, value):
        self.value = value
```

`__eq__(self, other)`:

Questo metodo viene chiamato quando viene utilizzato l'operatore di uguaglianza `==`. È possibile sovrascriverlo per definire il comportamento di confronto tra due oggetti della stessa classe.

```
pythonCopy code
def __eq__(self, other):
    return self.value == other.value
```

`__repr__(self)`:

Questo metodo restituisce la rappresentazione "ufficiale" di un oggetto come stringa. È spesso utilizzato per scopi di debugging e per fornire una rappresentazione completa dell'oggetto.

```
pythonCopy code
def __repr__(self):
    return f"MyClass(value={self.value})"
```

`__str__(self)`:

Questo metodo restituisce una rappresentazione "informale" dell'oggetto come stringa. È utilizzato quando viene chiamata la funzione `str()` su un oggetto.

```
pythonCopy code
def __str__(self):
    return f"Value: {self.value}"
```

`__lt__(self, other)`:

Questo metodo definisce il comportamento dell'operatore di confronto "minore di" `<`. È utilizzato per stabilire l'ordine tra due oggetti della classe.

```
pythonCopy code
def __lt__(self, other):
    return self.value < other.value
```

`__len__(self)`:

Il metodo `__len__` è utilizzato per definire la lunghezza di un oggetto. Viene chiamato quando la funzione built-in `len()` viene utilizzata su un'istanza di una classe. Questo metodo dovrebbe restituire la lunghezza dell'oggetto.

```
pythonCopy code
class MiaClasse:
```

```
    def __init__(self, elementi):
        self.elementi = elementi

    def __len__(self):
        return len(self.elementi)
```

`__getitem__(self, chiave)`:

Il metodo `__getitem__` consente di accedere alle istanze di una classe utilizzando la notazione delle parentesi quadre (`[]`). Viene chiamato quando si cerca di recuperare il valore associato a una specifica chiave o indice.

```
pythonCopy code
class MiaClasse:
    def __init__(self, dati):
        self.dati = dati

    def __getitem__(self, indice):
        return self.dati[indice]
```

`__setitem__(self, chiave, valore)`:

Il metodo `__setitem__` è il controparte di `__getitem__`. Viene chiamato quando si cerca di assegnare un valore a una specifica chiave o indice utilizzando la notazione delle parentesi quadre (`[]`).

```
pythonCopy code
class MiaClasse:
    def __init__(self, dati):
        self.dati = dati

    def __setitem__(self, indice, valore):
        self.dati[indice] = valore
```

`__eq__()`

Il metodo `__eq__()` viene utilizzato per definire il comportamento di confronto di uguaglianza tra due oggetti della stessa classe. Quando usi operatori di confronto come `==` o `!=` su istanze di una classe personalizzata, Python chiama implicitamente il metodo `__eq__()` per determinare se due oggetti sono considerati uguali o diversi.

```
class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def __eq__(self, other):
        if isinstance(other, Persona):
            return self.nome == other.nome and self.eta == ot
her.eta
        return False
```

`__hash__()`

Il metodo `__hash__()` viene utilizzato per generare un valore hash per un oggetto. Un valore hash è un numero intero che rappresenta in modo univoco l'oggetto. Questo è utile quando si desidera utilizzare gli oggetti come chiavi in strutture dati come dizionari o set.

```
class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def __eq__(self, other):
        if isinstance(other, Persona):
```

```
            return self.nome == other.nome and self.eta == ot
her.eta
        return False

    def __hash__(self):
        return hash((self.nome, self.eta))
```

Relazione tra `__eq__()` e `__hash__()`

Se implementi `__eq__()`, dovresti anche implementare `__hash__()`. Questo è importante perché gli oggetti che sono considerati uguali (cioè restituiscono `True` quando confrontati con `==`) devono avere lo stesso valore hash.

In altre parole, se due oggetti sono uguali, devono avere lo stesso hash, altrimenti potrebbero comportarsi in modo imprevisto quando vengono utilizzati come chiavi in strutture dati hash-based come dizionari o set.

Tuttavia, è importante notare che non è necessario implementare `__hash__()` per tutti gli oggetti. Se un oggetto è mutabile e il suo stato può cambiare dopo essere stato inserito in una struttura dati hash-based, non dovrebbe implementare `__hash__()`.
In questo caso, è meglio sollevare un'eccezione `TypeError` se si tenta di chiamare `hash()` su di esso. Questo perché gli oggetti mutabili non dovrebbero essere utilizzati come chiavi in strutture dati hash-based, poiché il loro valore hash potrebbe cambiare dopo essere stato inserito.